Passer du PHP à Node.js est une transition naturelle : vous connaissez déjà la logique serveur (requêtes, réponses, bases de données), mais vous allez découvrir une manière différente de gérer le temps (l'asynchronisme) et un langage unique pour le front et le back.

Pour un public **DWWM (Développeur Web et Web Mobile)**, la clé est de démystifier le côté "boîte noire" de Node.js en le comparant souvent à ce qu'ils connaissent en PHP.

🧭 Le Parcours d'Apprentissage

Nous allons avancer par modules. Chaque module contient : une explication théorique, un TP guidé, et une "Fiche Mémo".

Phase 1 : L'Environnement et les Fondamentaux

- **Concept :** Comprendre que Node.js n'est pas un langage, mais un moteur (V8) qui permet d'exécuter du JavaScript hors du navigateur.
- **Installation :** Utilisation de `nvm` (Node Version Manager) pour gérer les versions.
- **Le terminal :** Devenir ami avec la ligne de commande.

Phase 2 : NPM et l'Architecture d'un Projet

- **Concept :** Le fichier `package.json` (l'équivalent de `composer.json` en PHP).
- **Pratique :** Installer des dépendances, comprendre `node_modules`.

Phase 3 : Créer son premier serveur (Sans Framework)

- **Concept :** Comprendre le module `http` natif pour voir comment Node écoute sur un port.
- **Comparaison :** Pourquoi n'a-t-on pas besoin d'Apache ou Nginx en développement ?

Phase 4 : Express.js (Le "Standard" du Back-end)

- **Concept :** Le routage, les middlewares (notion cruciale en Node).
- **Réalisation :** Créer une API REST simple qui renvoie du JSON.

Phase 5 : Base de données et Asynchronisme

- **Concept :** `async / await` pour gérer les opérations longues sans bloquer le serveur.
- **Pratique :** Connexion à MongoDB ou MySQL.

🚀 Étape 1 : Installation et Premier Script

Pour commencer, installons Node.js proprement.

1. L'installation (Recommandation pédagogique) :

Ne téléchargez pas l'installateur classique. Apprenez à utiliser nvm (Node Version Manager). Cela permet de changer de version de Node en une commande.

2. Votre premier test :

Créez un dossier init-node, ouvrez un terminal dedans et tapez :

Bash

```bash
npm init -y
```

Cela crée votre `package.json`. Créez ensuite un fichier `app.js` :

JavaScript

```javascript
// app.js
const nom = "Apprenant DWWM";
console.log(`Bonjour ${nom}, Node fonctionne !`);
```

Lancez-le avec : `node app.js`.

📝 Fiche Pédagogique n°1

Node.js vs PHP : Ce qui change

- **PHP :** Un script s'exécute de haut en bas, puis "meurt" une fois la page envoyée. On a besoin d'un serveur web (Apache).
- **Node.js :** Le programme est un processus qui tourne en continu. Il attend des événements. Il est son propre serveur.
- **Points communs :** Gestion de requêtes HTTP, accès fichiers, connexion BDD.

🧠 Test de compréhension (Niveau 0)

Avant d'aller plus loin, essayez de répondre à ces trois questions (je vous donnerai la correction) :

1. Si je modifie mon code JS alors que le serveur tourne, les changements sont-ils pris en compte immédiatement ?
2. À quoi sert le dossier `node_modules` ?
3. Quelle commande permet d'installer une bibliothèque externe ?

🛠️ Ma proposition pour la suite

Je vous suggère de procéder ainsi : nous traitons une phase à la fois.

Je vous donne le contenu technique pour vous, puis je vous rédige la fiche pédagogique prête à l'emploi.

Pour bien comprendre Node.js, il faut d'abord oublier Apache ou Nginx. En Node.js, **c'est votre code qui crée le serveur**.

Voici la **Phase 3** : comprendre la mécanique de base avec le module natif `http`.

⚙️ 1. Pour vous : La théorie "Low Level"

En PHP, la superglobale `$_GET` ou `$_POST` est déjà là, prête à l'emploi. En Node.js, on reçoit un **flux de données** qu'il faut intercepter.

Le module `http` est intégré par défaut. Voici le code minimal pour créer un serveur :

JavaScript

```javascript
// On importe le module natif http
const http = require('http');

// On crée le serveur
const server = http.createServer((req, res) => {
    // req = request (ce que l'utilisateur envoie)
    // res = response (ce que nous renvoyons)

    // On définit l'en-tête de la réponse (Statut 200 = OK)
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
```

```
    // On écrit le corps de la réponse
    res.write('<h1>Hello DWWM !</h1>');
    res.write('<p>Ceci est un serveur Node sans framework.</p>');

    // On termine la réponse
    res.end();
});

// On dit au serveur d'écouter sur le port 3000
server.listen(3000, () => {
    console.log("Serveur démarré sur http://localhost:3000");
});
```

Comment tester ?

1. Enregistrez ce code dans un fichier `server.js`.
2. Dans votre terminal, lancez : `node server.js`.
3. Allez sur votre navigateur à l'adresse `http://localhost:3000`.

📘 2. La Fiche Pédagogique :

Voici une structure de fiche que vous pouvez copier-coller :

📝 Fiche : Créer son premier serveur Node.js

Objectif : Comprendre comment Node.js communique avec le navigateur.

1. Le concept du "Request / Response"

Contrairement au PHP où le serveur (Apache) est déjà là, en Node, c'est votre script qui écoute les requêtes.

- **Request (`req`) :** Contient l'URL, les cookies, les données du formulaire.
- **Response (`res`) :** C'est l'objet que l'on remplit pour répondre au client.

2. Les étapes clés du code :

- `require('http')` : On appelle l'outil de base.
- `createServer` : On définit ce qu'on répond quand quelqu'un se connecte.
- `res.writeHead` : On envoie le code HTTP et le type de contenu (HTML, JSON, etc.).
- `listen(3000)` : On ouvre la porte n°3000 de l'ordinateur pour laisser entrer les visiteurs.

⚠️ **Attention :** Si vous modifiez votre fichier `server.js`, vous devez arrêter le terminal (`Ctrl + C`) et relancer `node server.js` pour voir les changements !

🧠 Test de compréhension (Auto-correction)

Essayez de modifier votre code `server.js` pour réaliser cet exercice :

1. **Le défi :** Modifiez le serveur pour qu'il affiche "Page d'accueil" si l'URL est `/` et "Page Contact" si l'URL est `/contact`.
    - *Indice :* Utilisez la propriété `req.url`.
2. **Question :** Que se passe-t-il si vous tapez une URL différente (ex: `/test`) ?
3. **Analyse :** Pourquoi est-ce plus fastidieux qu'en PHP si on veut gérer 50 pages différentes de cette manière ?

Une fois que vous avez testé cela, nous passerons à l'étape suivante : automatiser le redémarrage (Nodemon) et utiliser Express pour simplifier tout ce code !

Voici la solution de l'exercice. C'est une étape cruciale car elle montre la difficulté de gérer les "routes" à la main, ce qui justifiera plus tard l'utilisation d'Express.js.

🛠️ La Solution : Gestion des routes natives

Modifiez votre fichier `server.js` avec ce code :

JavaScript

```javascript
const http = require('http');

const server = http.createServer((req, res) => {
    // On définit l'encodage pour éviter les problèmes d'accents
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });

    // Analyse de l'URL demandée
    if (req.url === '/') {
        res.write('<h1>Bienvenue sur la Page d\'accueil</h1>');
    }
    else if (req.url === '/contact') {
```

```
        res.write('<h1>Page Contact</h1><p>Écrivez-nous à contact@dev.fr</
p>');
    }
    else {
        // Gestion de l'erreur 404 (si l'URL ne correspond à rien)
        res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8' });
        res.write('<h1>404 - Page non trouvée</h1>');
    }

    res.end();
});

server.listen(3000, () => {
    console.log("Serveur en écoute sur http://localhost:3000");
});
```

🧐 Analyse des réponses

1. **Le défi (Le code) :** Comme vous le voyez, on utilise une structure `if / else if` basée sur `req.url`. C'est l'ancêtre du système de "Routing".
2. **Si on tape `/test` ?** Le code tombe dans le dernier `else`. Sans cela, le navigateur chargerait à l'infini car le serveur n'aurait jamais reçu l'ordre `res.end()`.
3. **Pourquoi est-ce fastidieux ?** En PHP, il suffit de créer un fichier `contact.php` et Apache fait le lien tout seul. Ici, si vous avez 50 pages, votre `if / else` va devenir illisible. C'est pour cela qu'on utilise des **Frameworks comme Express.js**.

📦 Étape 4 : Faciliter la vie avec Nodemon et Express

Avant de passer à Express, nous allons régler un problème agaçant : devoir relancer le serveur à chaque modification.

1. Nodemon (L'outil indispensable)

C'est un utilitaire qui surveille vos fichiers. S'ils changent, il relance le serveur pour vous.

Installation globale :

Bash

```
npm install -g nodemon
```

Note : -g signifie "partout sur l'ordinateur".

Usage : Au lieu de faire node server.js, faites nodemon server.js.

2. Passer à Express.js

Express est à Node ce que Symfony ou Laravel sont au PHP (en plus léger). C'est lui que vous enseignerez majoritairement en DWWM.

Installation dans votre dossier projet :

Bash

```
npm install express
```

Le même serveur, mais avec Express (`app.js`) :

JavaScript

```javascript
const express = require('express');
const app = express();
const port = 3000;

// Route pour l'accueil
app.get('/', (req, res) => {
    res.send('<h1>Bienvenue avec Express !</h1>');
});

// Route pour le contact
app.get('/contact', (req, res) => {
    res.send('<h1>Page Contact</h1>');
});

app.listen(port, () => {
    console.log(`Application lancée sur http://localhost:${port}`);
});
```

📘 Fiche Pédagogique n°2 : Express.js & NPM

📝 Fiche : Simplifier le Backend avec Express

NPM (Node Package Manager) : C'est le magasin d'outils. On utilise `npm install <nom>` pour ajouter des fonctionnalités (ex: Express, gestion des images, connexion BDD).

Pourquoi Express ?

1. **Code plus propre :** On remplace les `if/else` par des méthodes claires (`app.get`, `app.post`).
2. **Abstraction :** Express gère automatiquement les types de contenu (Content-Type) et les erreurs de base.
3. **Standard du marché :** La majorité des entreprises utilisent Express ou un dérivé.

Commande à retenir : > * `npm init` : Initialiser un projet.

- `npm install express` : Ajouter le framework au projet.

🧠 Test de compréhension pour vous

1. Dans le code Express ci-dessus, j'utilise `res.send()` au lieu de `res.write()` + `res.end()`. À votre avis, quel est l'avantage ?
2. Comment s'appelle le dossier où NPM télécharge les fichiers d'Express ?
3. Si je veux créer une route `/api/data` qui renvoie du texte, quelle ligne de code dois-je ajouter ?

On monte d'un cran. Maintenant que vous avez un serveur qui répond, nous allons voir comment **récupérer des informations** de l'utilisateur.

En DWWM, c'est une étape clé : c'est ici qu'on apprend à créer des routes dynamiques (ex: afficher le profil d'un utilisateur spécifique).

🛠️ 1. Pour vous : Paramètres et JSON

Dans le monde des API et de Node.js, on ne renvoie pas toujours du HTML. On renvoie souvent du **JSON**. Express rend cela extrêmement simple.

Les deux types de récupération de données :

1. **Les Paramètres d'URL (`req.params`)** : Pour les identifiants (ex: `/utilisateurs/42`).
2. **Les Query Strings (`req.query`)** : Pour les filtres (ex: `/recherche?couleur=bleu`).

Voici le code à tester dans votre `app.js` :

JavaScript

```
const express = require('express');
const app = express();

// 1. Route avec paramètre dynamique (:id)
app.get('/user/:id', (req, res) => {
    const userId = req.params.id;
    res.send(`<h1>Profil de l'utilisateur n°${userId}</h1>`);
});

// 2. Route qui renvoie du JSON (format API)
app.get('/api/data', (req, res) => {
    const formation = {
        titre: "DWWM",
        langage: "Node.js",
        difficulte: "Intermédiaire"
    };
    // res.json() convertit automatiquement l'objet en texte JSON
    res.json(formation);
});

app.listen(3000, () => console.log("Prêt sur http://localhost:3000"));
```

📘 2. La Fiche Pédagogique : Les Routes Dynamiques

📝 Fiche : Récupérer des données dans l'URL

Le concept :

On ne peut pas créer une route pour chaque utilisateur (il y en a des milliers !). On utilise donc des variables de route.

Syntaxe :

- Dans le code : app.get('/produit/:nom' ...)
- Dans le navigateur : localhost:3000/produit/clavier
- Récupération : req.params.nom contiendra la valeur "clavier".

JSON vs HTML :

- res.send() : Envoie principalement du texte ou du HTML (pour les sites classiques).

- `res.json()` : Envoie des données structurées. C'est le format standard pour que le Front-end (React, Vue) communique avec le Back-end.

🚀 3. Le concept crucial : Le Middleware

C'est ici que Node.js diffère un peu du PHP classique. Un **middleware**, c'est une fonction qui s'exécute **entre** la réception de la requête et l'envoi de la réponse.

Imaginez un videur de boîte de nuit : il vérifie votre identité avant de vous laisser entrer.

Exemple de réalisation commentée :

Nous allons créer un petit script qui affiche l'heure à chaque fois qu'une page est visitée.

JavaScript

```
// Définition du Middleware
const loggerMiddleware = (req, res, next) => {
    console.log(`[${new Date().toLocaleTimeString()}] Requête reçue sur : ${req.url}`);
    next(); // INDISPENSABLE : dit à Express de passer à la suite
};

// On dit à Express d'utiliser ce middleware pour TOUTES les routes
app.use(loggerMiddleware);
```

🧠 Test de compréhension (TP Guidé)

Votre mission :

1. Créez une route `/calcul/:nombre1/:nombre2`.
2. Cette route doit récupérer les deux nombres dans l'URL.
3. Elle doit renvoyer un objet JSON contenant le résultat de l'addition.

Exemple attendu : Si je tape `localhost:3000/calcul/5/10`, je reçois `{"resultat": 15}`.

Petit piège : Les paramètres récupérés dans `req.params` sont toujours des **chaînes de caractères (Strings)**. Pensez à `Number()` pour les transformer !

Est-ce que vous réussissez à faire fonctionner ce petit "calculateur" ?

Pas de panique ! C'est tout à fait normal. Quand on vient du PHP, on a l'habitude de manipuler des tableaux (array), alors qu'en Node.js, on manipule des **Objets**.

Voici le code détaillé et commenté pour réussir cette route, ainsi que l'explication du "pourquoi" ça bloque parfois.

🛠️ La solution commentée : La route /api/data

Placez ce code dans votre fichier app.js (assurez-vous d'avoir bien fait const app = express(); au début).

JavaScript

```
// La route API qui renvoie du JSON
app.get('/api/data', (req, res) => {
    // 1. On crée un objet JavaScript (équivalent d'un tableau associatif en PHP)
    const monResultat = {
        statut: "Succès",
        message: "Voici les données demandées",
        cours: ["Node.js", "Express", "JavaScript"],
        nbrApprenants: 15
    };

    // 2. On utilise res.json()
    // Cette méthode fait deux choses :
    // - Elle transforme l'objet en texte (String) au format JSON
    // - Elle dit au navigateur : "Attention, je t'envoie du JSON, pas du HTML"
    res.json(monResultat);
});
```

💡 Pourquoi ça n'a peut-être pas marché ? (Checklist)

1. **L'ordre des routes :** Vérifiez que vous n'avez pas une route /api/:quelquechose placée **au-dessus**. Node.js lit les routes de haut en bas. S'il trouve une correspondance, il s'arrête.

2. **L'oubli du `res.json` :** Si vous faites `res.send(monResultat)`, Express essaiera de deviner le type, mais `res.json()` est plus explicite et sécurisé pour une API.
3. **Le redémarrage :** Si vous n'utilisez pas `nodemon`, avez-vous bien coupé le terminal (`Ctrl+C`) et relancé `node app.js` ?

🏗️ Le TP "Calculateur" (La solution)

Pour l'exercice du calculateur `/calcul/:nombre1/:nombre2`, voici comment combiner les **paramètres** et le **JSON** :

JavaScript

```
app.get('/calcul/:nombre1/:nombre2', (req, res) => {
    // Récupération des paramètres (attention : ce sont des chaînes de caractères !)
    const n1 = Number(req.params.nombre1);
    const n2 = Number(req.params.nombre2);

    // Calcul
    const somme = n1 + n2;

    // Réponse en JSON
    res.json({
        operation: "addition",
        nombre1: n1,
        nombre2: n2,
        resultat: somme
    });
});
```

📘 Fiche Pédagogique n°3 : Le format JSON et les API

📝 Fiche : Communiquer en JSON

C'est quoi le JSON ?

Le JavaScript Object Notation est le langage universel pour échanger des données. C'est du texte qui ressemble à un objet JavaScript.

Pourquoi l'utiliser en DWWM ?

Aujourd'hui, on sépare souvent le Back-end (Node.js) du Front-end (React, Vue, ou simple JS). Le Back n'envoie plus de HTML tout fait, il envoie des données brutes (JSON) que le Front se charge d'afficher.

La différence de syntaxe :

- **PHP :** `echo json_encode($monTableau);`
- **Node/Express :** `res.json(monObjet);`

Points de vigilance :

- Les données reçues dans l'URL (`req.params`) sont toujours des **Strings**. Il faut les convertir avec `Number()` ou `parseInt()` avant de faire des maths !

Est-ce que cette fois la route `/api/data` s'affiche bien dans votre navigateur ? Si oui, nous pourrons passer à une étape cruciale : **L'organisation du code (les dossiers)** pour éviter d'avoir un fichier `app.js` de 500 lignes !

C'est une excellente question qui touche à l'organisation des fichiers. La réponse courte est : **cela dépend du nom que vous avez donné à votre fichier principal.**

En PHP, vous avez l'habitude d'avoir un fichier par page (`index.php`, `contact.php`). En Node.js, on a généralement **un seul point d'entrée** (le serveur) qui gère toutes les routes.

1. Clarification : `server.js` ou `app.js` ?

Si au début vous avez créé un fichier nommé `server.js` pour faire vos tests, et que maintenant vous écrivez du code Express dans un fichier nommé `app.js`, **le terminal ne devine pas le changement.**

- Si vous lancez `node server.js`, il exécutera l'ancien code (sans Express).
- Si vous voulez utiliser Express et votre nouvelle route /api/data, vous devez lancer : node app.js (ou nodemon app.js).

Le plus propre est de tout regrouper dans un seul fichier pour l'instant. Voici à quoi devrait ressembler votre fichier complet (appelons-le `app.js`) :

JavaScript

```
const express = require('express'); // Importation
const app = express();              // Initialisation

// --- Vos Routes ---
```

```
app.get('/', (req, res) => {
    res.send('<h1>Accueil</h1>');
});

// La fameuse route qui vous posait problème
app.get('/api/data', (req, res) => {
    res.json({ message: "Ça marche enfin !", statut: 200 });
});

app.get('/calcul/:nombre1/:nombre2', (req, res) => {
    const somme = Number(req.params.nombre1) +
Number(req.params.nombre2);
    res.json({ resultat: somme });
});

// --- Lancement du serveur ---
app.listen(3000, () => {
    console.log("Serveur actif sur http://localhost:3000");
});
```

2. Pourquoi on finit par séparer les fichiers ?

Vous allez vite apprendre que mettre 50 routes dans le même fichier est illisible.
On utilise alors le **Router d'Express**.

Imaginez une armoire :

- `app.js` est l'armoire (le serveur).
- Chaque tiroir est un fichier de "Routes" (ex: `routes/users.js`, `routes/api.js`).

📘 Fiche Pédagogique n°4 : Le point d'entrée unique

📝 Fiche : L'architecture d'un projet Node

Le fichier "Main" (Point d'entrée) :

En Node.js, on définit un fichier principal (souvent app.js ou index.js). C'est lui qui "allume" le serveur.

- Si vous ajoutez une route dans le code mais que vous ne relancez pas **ce fichier précis**, la route n'existera pas pour le navigateur.

L'analogie avec PHP :

- En PHP classique : `site.com/contact.php` cherche réellement un fichier physique sur le disque.

- En Node.js : site.com/contact cherche une **fonction** définie dans votre code. Le fichier contact.php n'existe pas, c'est une "route virtuelle".

Astuce Dépanage :

Si une route renvoie une erreur 404 alors que vous l'avez écrite :

1. Vérifiez que vous avez enregistré le fichier.
2. Vérifiez que le terminal fait tourner le bon fichier.
3. Vérifiez l'ordre des routes (le serveur lit de haut en bas).

🧠 Test de compréhension

Pour vérifier que vous maîtrisez le lien entre les fichiers :

1. Si vous créez un nouveau fichier test.js avec une route, et que vous tapez node app.js dans le terminal, est-ce que la route de test.js fonctionnera ?
2. Quelle commande npm permet de voir toutes les dépendances installées (comme Express) dans votre projet ? (Indice : regardez le fichier créé au tout début).

C'est une excellente question qui touche au concept de **Middleware**. L'emplacement dépend de la portée que vous voulez donner à ce script (est-ce pour toutes les pages ou juste une seule ?).

Pour un script qui affiche l'heure à **chaque** visite, on le place juste après avoir initialisé l'application, mais **avant** la définition de vos routes.

🛠️ L'emplacement dans votre code (app.js)

Voici l'ordre logique et obligatoire pour que cela fonctionne :

JavaScript

```
const express = require('express');
const app = express();

// --- 1. PLACEMENT DU MIDDLEWARE (L'HEURE) ---
// On le met AVANT les routes pour qu'il intercepte tout
app.use((req, res, next) => {
    const date = new Date().toLocaleTimeString();
    console.log(`[${date}] Quelqu'un a visité : ${req.url}`);
```

```
    // IMPORTANT : next() permet de passer à la route suivante.
    // Sans lui, le navigateur resterait bloqué sur une page blanche !
    next();
});

// --- 2. VOS ROUTES ---
app.get('/', (req, res) => {
    res.send('Accueil');
});

app.get('/api/data', (req, res) => {
    res.json({ message: "Données" });
});

app.listen(3000);
```

🤔 Pourquoi à cet endroit ?

Node.js lit le code **de haut en bas**.

- Si vous placez le script *après* vos routes, il ne sera jamais exécuté car la réponse aura déjà été envoyée au navigateur par `res.send()` ou `res.json()`.
- En le plaçant en haut avec `app.use()`, vous créez un "entonnoir" par lequel toutes les requêtes doivent passer avant d'atteindre leur destination.

📘 Fiche Pédagogique n°5 : Le Middleware (Le "Passage Obligé")

📝 Fiche : Comprendre les Middlewares

Définition :

Un middleware est une fonction intermédiaire qui a accès aux objets Requête (req) et Réponse (res).

Le rôle de next() :

C'est le passage de témoin. En PHP, le script est linéaire. En Node/Express, on peut avoir plusieurs fonctions qui se passent la main. Si vous oubliez next(), la requête s'arrête là et le visiteur attend indéfiniment.

Cas d'usage fréquents en DWWM :

1. **Logger :** Afficher l'heure et l'IP des visiteurs (ce que nous venons de faire).
2. **Sécurité :** Vérifier si un utilisateur est connecté avant d'afficher une page "Mon Compte".

3. **Parsing :** Analyser les données envoyées par un formulaire (on verra cela bientôt avec express.json()).

Schéma mental :

Client ➡️ Middleware (Log/Heure) ➡️ Middleware (Sécurité) ➡️ Route (Page demandée) ➡️ Réponse au Client.

🧠 Test de compréhension

1. Que se passe-t-il concrètement dans votre terminal si vous retirez la ligne next(); du middleware de l'heure et que vous rafraîchissez la page dans votre navigateur ?
2. Si je veux que l'heure ne s'affiche **que** pour la route /api/data et pas pour l'accueil, comment devrais-je réorganiser mon code ? (Indice : on peut passer la fonction directement dans la route).

Voulez-vous essayer de créer un middleware de sécurité simple (par exemple : si l'URL ne contient pas un mot de passe, on refuse l'accès) ?

Si l'heure n'apparaît pas dans votre console (le terminal), c'est généralement dû à l'un de ces trois facteurs. Vérifions cela ensemble, car c'est un excellent cas d'école sur le "débogage" Node.js.

🔍 Les 3 causes probables

1. L'oubli de rafraîchir la page

Le console.log ne s'exécute que lorsqu'une **requête** arrive au serveur.

- Lancez votre serveur (node app.js).
- Ouvrez votre navigateur sur http://localhost:3000.
- **Regardez votre terminal** (pas la console du navigateur !) juste après avoir rafraîchi la page.

2. L'ordre du code (Le plus fréquent)

Si vous avez placé le app.use **après** vos routes app.get, il ne s'exécutera jamais.

- **Pourquoi ?** Car la route app.get('/') envoie une réponse et termine le cycle. Le code situé en dessous n'est jamais lu pour cette requête.

3. Le terminal n'est pas "à jour"

Si vous avez ajouté le code mais que vous n'avez pas arrêté (Ctrl+C) et relancé le serveur, Node.js exécute toujours l'ancienne version de votre script (celle qui n'avait pas le log).

🛠️ Le Code "Zéro Erreur" à tester

Copiez-collez exactement ceci dans votre fichier pour être sûr :

JavaScript

```
const express = require('express');
const app = express();

// 1. LE MIDDLEWARE DOIT ÊTRE ICI (En premier !)
app.use((req, res, next) => {
    const heure = new Date().toLocaleTimeString();
    console.log(`--- PASSAGE DANS LE MIDDLEWARE ---`);
    console.log(`Heure : ${heure} | Route appelée : ${req.url}`);
    next();
});

// 2. LES ROUTES VIENNENT APRÈS
app.get('/', (req, res) => {
    res.send("Regardez votre terminal (l'invite de commande) !");
});

app.listen(3000, () => {
    console.log("Le serveur tourne. En attente de visites sur http://
localhost:3000 ...");
});
```

📘 Fiche Pédagogique n°6 : Où regarder quand on développe ?

📝 Fiche : Terminal vs Console Navigateur

En développement Web Mobile (DWWM), il ne faut pas confondre les deux outils de débogage :

1. La Console du Navigateur (F12) :

- Elle affiche les erreurs de votre **Front-end** (JavaScript client, CSS, HTML).
- On y voit : console.log écrits dans les fichiers chargés par la page HTML.

2. Le Terminal (L'invite de commande) :

- Il affiche les messages de votre **Back-end** (Node.js).

- On y voit : Les erreurs de base de données, les ports occupés, et les console.log écrits dans vos fichiers .js côté serveur (comme app.js).

Règle d'or : En Node.js, si vous écrivez console.log, c'est dans le **terminal** que la réponse apparaîtra, JAMAIS dans le navigateur.

🧠 Test rapide pour vous

1. Avez-vous bien deux messages dans votre terminal au lancement : celui du listen ET celui du logger (quand vous visitez la page) ?
2. Si vous visitez http://localhost:3000/api/data, est-ce que le terminal affiche bien /api/data dans le log ?

Est-ce que l'heure s'affiche enfin ? Si oui, nous pourrons attaquer la partie "Formulaires" : comment récupérer des données envoyées via une méthode **POST** (comme un formulaire d'inscription).

Protéger des routes. En DWWM, c'est ce qu'on appelle une **barrière d'authentification**.

🛠️ 1. Pour vous : Le Middleware de "Sécurité"

Nous allons utiliser une **Query String** (ce qui se trouve après le ? dans l'URL). L'idée est la suivante : si l'utilisateur n'ajoute pas ?password=123 à la fin de l'URL, on lui bloque l'accès.

Voici comment modifier votre fichier app.js :

JavaScript

```
const express = require('express');
const app = express();

// --- LE MIDDLEWARE DE SÉCURITÉ ---
const securityCheck = (req, res, next) => {
    const password = req.query.password; // On récupère ce qui est après ?
password=

    if (password === '123') {
        console.log("Accès autorisé ✅");
        next(); // On laisse passer à la route suivante
    } else {
```

```
        console.log("Accès refusé ❌");
        res.status(403).send("<h1>Accès refusé</h1><p>Mot de passe incorrect ou absent.</p>");
        // Pas de next() ici, donc la requête s'arrête net !
    }
};

// --- LES ROUTES ---

app.get('/', (req, res) => {
    res.send("Accueil libre d'accès.");
});

// On applique le middleware UNIQUEMENT à cette route "secrète"
app.get('/admin', securityCheck, (req, res) => {
    res.send("<h1>Espace Admin</h1><p>Bienvenue, maître du monde.</p>");
});

app.listen(3000);
```

🧪 Comment tester ?

1. Allez sur `http://localhost:3000/admin`.
    - Résultat : **Accès refusé**.
2. Allez sur `http://localhost:3000/admin?password=123`.
    - Résultat : **Espace Admin** s'affiche !

📘 2. La Fiche Pédagogique : Sécuriser les accès

📝 Fiche : Authentification et Code HTTP 403

Le concept :

Un middleware peut agir comme un filtre. Il examine la requête et décide si elle peut continuer son chemin vers la route finale.

L'objet req.query :

En Node.js, tout ce qui est écrit après le ? dans l'URL est automatiquement rangé dans un objet appelé req.query.

- Exemple : `/search?color=red&size=m` devient `{ color: 'red', size: 'm' }`.

Les codes de statut HTTP :

Il est important d'indiquer au navigateur (et aux moteurs de recherche) ce qu'il se passe :

- `200` : OK (Tout s'est bien passé).
- `403` : Forbidden (Accès refusé).
- `404` : Not Found (La page n'existe pas).
- `500` : Internal Server Error (Votre code Node a planté).

Syntaxe Express : `res.status(403).send("Message");`

🧠 Test de compréhension

1. **Pourquoi** l'accueil (`/`) reste-t-il accessible même sans mot de passe ?
2. Dans le code ci-dessus, que se passe-t-il si j'oublie de mettre la ligne `next()` dans le bloc `if` ?
3. **Défi :** Comment modifieriez-vous le code pour que le mot de passe soit "azerty" au lieu de "123" ?

C'est une grande étape ! Une fois que vous êtes à l'aise avec ça, nous pourrons voir comment traiter les données d'un vrai formulaire HTML (méthode POST), car c'est ainsi que les utilisateurs s'inscrivent vraiment. On continue sur le POST ?

Passons à la méthode **POST**. C'est une étape majeure pour vos apprenants DWWM car c'est ainsi que l'on gère l'envoi de données sensibles (mots de passe, textes longs) qui ne doivent pas apparaître dans l'URL.

🛠️ 1. Pour vous : Le traitement du corps de la requête (`body`)

Contrairement au PHP où `$_POST` est rempli automatiquement, Node.js a besoin d'un petit coup de pouce (un middleware intégré) pour lire les données envoyées par un formulaire.

Étape 1 : Configurer Express pour lire le POST

Ajoutez cette ligne tout en haut de votre fichier, juste après avoir créé `const app = express();` :

JavaScript

```
app.use(express.urlencoded({ extended: true })); // Pour lire les formulaires classiques (HTML)
app.use(express.json()); // Pour lire le JSON (si vous utilisez une API
```

Étape 2 : Créer le formulaire et la route de réception

Voici un exemple complet que vous pouvez tester :

JavaScript

```javascript
const express = require('express');
const app = express();

app.use(express.urlencoded({ extended: true }));

// 1. Afficher le formulaire (GET)
app.get('/inscription', (req, res) => {
    res.send(`
        <h1>Inscription</h1>
        <form method="POST" action="/inscription">
            <input type="text" name="pseudo" placeholder="Votre pseudo" required>
            <input type="password" name="motdepasse" placeholder="Votre mot de passe" required>
            <button type="submit">S'inscrire</button>
        </form>
    `);
});

// 2. Recevoir les données (POST)
app.post('/inscription', (req, res) => {
    // Les données arrivent dans req.body grâce au middleware du haut
    const pseudo = req.body.pseudo;
    const mdp = req.body.motdepasse;

    console.log(`Nouvelle inscription : ${pseudo}`);

    res.send(`Merci ${pseudo}, votre inscription est reçue !`);
});
```

```
app.listen(3000);
```

📘 2. La Fiche Pédagogique : GET vs POST

📝 Fiche : Envoyer des données au serveur

La méthode GET :

- Les données passent par l'URL (`?id=5`).
- Limité en taille.
- Visible dans l'historique (🚫 jamais pour les mots de passe !).
- Récupération en Node : `req.query`.

La méthode POST :

- Les données passent par le "corps" (body) de la requête.
- Invisible dans l'URL.
- Pas de limite de taille.
- Récupération en Node : `req.body`.

Le Middleware express.urlencoded :

C'est le traducteur obligatoire. Sans lui, req.body sera vide (undefined). Il transforme le texte brut envoyé par le navigateur en un bel objet JavaScript facile à utiliser.

🧠 Test de compréhension (Exercice final de ce module)

Pour valider que vous avez bien compris le lien entre le Front et le Back :

1. **Question :** Si dans mon code HTML je change `<input name="pseudo">` par `<input name="username">`, que dois-je modifier dans ma route `app.post` ?
2. **Pratique :** Créez une route POST `/login`. Si le pseudo est "admin" et le mot de passe "123", renvoyez "Bienvenue chef !", sinon renvoyez "Identifiants incorrects".
3. **Analyse :** Pourquoi est-il impossible de tester une route `app.post` simplement en tapant l'adresse dans la barre du navigateur ?

Sortir le code HTML du fichier JavaScript. En PHP, on mélange souvent les deux, mais en Node.js, on sépare strictement la logique (le serveur) des fichiers visuels (le front).

Pour cela, on utilise le concept de **"Dossier Statique"**.

🛠️ 1. Pour vous : Servir des fichiers statiques

Express possède un middleware intégré très puissant : `express.static`. Il permet de désigner un dossier (souvent nommé `public`) dont le contenu sera directement accessible par le navigateur.

Étape 1 : Organiser vos dossiers

Créez la structure suivante dans votre projet :

Plaintext

```
mon-projet/
├── app.js
├── package.json
└── public/          <-- Tout le front va ici
    ├── index.html
    ├── style.css
    └── script.js
```

Étape 2 : Configurer `app.js`

Il suffit d'ajouter cette ligne (généralement placée avec les autres middlewares en haut) :

JavaScript

```javascript
const express = require('express');
const path = require('path'); // Module natif pour gérer les chemins de fichiers
const app = express();

// On dit à Express : "Si tu reçois une demande pour un fichier, regarde dans le dossier public"
app.use(express.static('public'));

app.listen(3000, () => console.log("Serveur sur http://localhost:3000"));
```

Étape 3 : Créer un fichier HTML (`public/index.html`)

HTML

```html
<!DOCTYPE html>
<html>
<head>
   <link rel="stylesheet" href="style.css"> </head>
<body>
   <h1>Bienvenue sur mon site Node.js !</h1>
   <p>Ce fichier est servi statiquement.</p>
</body>
</html>
```

Désormais, quand vous allez sur http://localhost:3000, Express cherche automatiquement un fichier index.html dans le dossier public.

📘 2. La Fiche Pédagogique : Les Fichiers Statiques

📝 Fiche : Séparer le Fond de la Forme

C'est quoi un fichier "Statique" ?

C'est un fichier qui ne change pas (HTML, CSS, Images, JS client). Le serveur Node n'a pas besoin de le "calculer", il a juste besoin de l'envoyer tel quel au navigateur.

L'analogie avec PHP :

En PHP, votre serveur Apache servait tout par défaut. En Node.js, par sécurité, rien n'est accessible par défaut. Il faut donner l'autorisation explicite via app.use(express.static('public')).

Pourquoi le dossier public ?

C'est une convention. On y met tout ce que le visiteur a le droit de voir. Vos fichiers serveurs (app.js, mots de passe de BDD) restent en dehors de ce dossier pour être protégés.

Le rôle du module path :

On l'utilise souvent pour s'assurer que le chemin vers le dossier est correct, quel que soit l'ordinateur (Windows ou Linux) où tourne le serveur.

🧠 Test de compréhension

1. **Question :** Si je place une image `logo.png` dans le dossier `public/images/`, quelle sera l'URL pour l'afficher dans mon navigateur ?
2. **Analyse :** J'ai mis mon fichier `secret.txt` à la racine de mon projet (à côté de `app.js`). Est-ce que quelqu'un peut le voir en tapant `localhost:3000/secret.txt` ? Pourquoi ?
3. **Pratique :** Essayez de créer un fichier CSS simple dans `public/style.css` (mettez le fond en bleu) et vérifiez qu'il s'applique bien à votre `index.html`.

Connecter tout cela à une Base de Données (MongoDB ou MySQL) pour rendre votre site vraiment dynamique ? C'est le moment où on commence à faire de la vraie gestion de données.

La **Phase 5** est le cœur du métier de Développeur Web (DWWM). Nous allons connecter notre serveur à une base de données.

Pour cet apprentissage, je vous propose d'utiliser **MongoDB** avec la bibliothèque **Mongoose**.

- **Pourquoi ?** C'est l'approche la plus courante avec Node.js (la pile MERN/MEAN). Contrairement au SQL, on stocke des objets qui ressemblent à du JSON, ce qui est beaucoup plus intuitif pour des débutants en JavaScript.

🛠️ 1. Pour vous : L'Asynchronisme et Mongoose

En PHP, quand vous faites une requête SQL, le script s'arrête et attend la réponse. En Node.js, on utilise `async` et `await` pour ne pas bloquer le serveur pendant que la base de données travaille.

Étape 1 : Installation

Dans votre terminal :

Bash

```
npm install mongoose
```

Étape 2 : Connexion et Modèle

Dans Node, on définit un **Schema** (la structure de nos données) avant d'écrire dedans.

JavaScript

```javascript
const mongoose = require('mongoose');

// 1. Connexion (Remplacez par votre lien MongoDB Atlas ou local)
mongoose.connect('mongodb://localhost:27017/ma_base_dwwm')
    .then(() => console.log("Connecté à MongoDB !"))
    .catch(err => console.error("Erreur de connexion", err));

// 2. Création du Schéma (Le "moule")
const userSchema = mongoose.Schema({
    pseudo: String,
    email: { type: String, required: true },
    age: Number
});

// 3. Création du Modèle (L'outil pour interagir avec la collection)
const User = mongoose.model('User', userSchema);
```

Étape 3 : Utiliser la base dans une route

Voici comment enregistrer un utilisateur via une route POST :

JavaScript

```javascript
app.post('/register', async (req, res) => {
    try {
        const novelUser = new User({
            pseudo: req.body.pseudo,
            email: req.body.email,
            age: req.body.age
        });

        await novelUser.save(); // On attend que l'enregistrement soit fini
        res.status(201).send("Utilisateur enregistré avec succès !");
    } catch (error) {
        res.status(400).send("Erreur lors de l'enregistrement");
    }
});
```

📘 2. La Fiche Pédagogique : Base de Données NoSQL

📝 Fiche : Node.js + MongoDB (Le couple idéal)

SQL vs NoSQL :

- **SQL (MySQL) :** Tables rigides, relations complexes (JOIN). C'est comme un classeur avec des intercalaires fixes.
- **NoSQL (MongoDB) :** Documents flexibles (JSON). C'est comme une pile de dossiers où chaque feuille peut être un peu différente.

L'Asynchronisme (async / await) :

C'est la notion la plus importante. Communiquer avec une base de données prend du temps (quelques millisecondes).

- `async` : Dit à Node que la fonction contient une opération longue.
- `await` : Dit à Node "Attends ici que la base de données réponde avant de passer à la ligne suivante".

Mongoose :

C'est l'outil (ODM) qui fait le pont entre votre code JavaScript et la base MongoDB. Il permet de valider les données (ex: vérifier qu'un email est bien présent).

🧠 Test de compréhension

1. **Question :** Pourquoi utilise-t-on `try { ... } catch (error) { ... }` autour de l'enregistrement en base de données ?
2. **Logique :** Si j'ai 1000 utilisateurs en base et que je veux tous les récupérer, quelle méthode Mongoose devrais-je utiliser selon vous : `User.find()` ou `User.save()` ?
3. **Analyse :** En PHP, que se passerait-il si la base de données mettait 10 secondes à répondre ? Et en Node.js avec `await` ?

🏁 Réalisation finale : Votre premier projet complet

Pour valider tout ce que nous avons vu (Installation, Express, Routes, Statique, Middleware, POST, BDD), je vous propose de réaliser ce mini-projet :

"Le Livre d'Or"

1. Une page HTML avec un formulaire (Pseudo + Message).
2. Une route POST qui enregistre le message dans MongoDB.
3. Une route GET qui affiche la liste de tous les messages postés sous forme de JSON (ou HTML).

Comprendre la différence entre les **méthodes HTTP**.

🔍 Pourquoi ce message apparaît ?

L'erreur `Cannot GET /register` signifie que vous essayez d'accéder à la page en tapant l'adresse directement dans votre navigateur, mais que dans votre code, vous n'avez défini cette route qu'avec **`app.post`**.

1. **Le Navigateur :** Quand vous tapez une URL et validez, il effectue toujours une requête **GET**.
2. **Votre Code :** Vous avez écrit `app.post('/register', ...)`. Votre serveur attend donc que des données lui soient "poussées" (via un formulaire ou une API), et non qu'on lui demande simplement d'afficher la page.

🛠️ La Solution : Créer les deux "portes"

Pour que cela fonctionne, vous devez souvent créer deux routes avec le même nom, mais des méthodes différentes :

JavaScript

```
// 1. La porte d'entrée (GET) : pour afficher le formulaire
app.get('/register', (req, res) => {
    res.send(`
        <h1>Inscription</h1>
        <form action="/register" method="POST">
            <input type="text" name="pseudo" placeholder="Pseudo">
            <input type="email" name="email" placeholder="Email">
            <button type="submit">S'inscrire</button>
        </form>
    `);
});

// 2. La porte de réception (POST) : pour traiter les données
app.post('/register', async (req, res) => {
    // ... votre code Mongoose pour enregistrer l'utilisateur ...
    res.send("Données reçues !");
});
```

📘 Fiche Pédagogique n°7 : Le Routage et les Méthodes

📝 Fiche : Comprendre les erreurs "Cannot GET"

Le problème :

Une route est définie par deux éléments : son URL (ex: /register) ET sa Méthode (GET, POST, PUT, DELETE).

- Si vous définissez `app.post('/register')` et que vous visitez l'URL dans le navigateur, Express répond `Cannot GET` car il ne trouve pas de correspondance pour la méthode GET.

Les bonnes pratiques (REST) :

- **GET /utilisateurs** : On demande à voir la liste.
- **POST /utilisateurs** : On demande à créer un nouvel utilisateur.
- **GET /utilisateurs/1** : On demande à voir le détail du n°1.

Astuce Débogage :

Toujours vérifier dans le code si la méthode utilisée par le client (Formulaire ou Navigateur) correspond bien à la fonction app.METHOD écrite dans Node.js.

🧠 Test de compréhension

1. Si je veux modifier mon formulaire pour qu'il ne cause plus l'erreur `Cannot GET`, mais que je ne veux pas rajouter de code dans `app.js`, est-ce que changer `<form method="POST">` en `<form method="GET">` fonctionnerait ? (Indice : regardez le nom de votre route dans le serveur).
2. Dans un projet DWWM, quel outil utilise-t-on souvent pour tester des routes `POST` ou `PUT` sans avoir à créer de formulaire HTML à chaque fois ? (Indice : c'est un logiciel dont le nom commence par **P** ou une extension VS Code).

allons créer un **Livre d'Or** complet. Ce projet est parfait car il couvre le cycle complet des données : **Saisie (Front) -> Envoi (Route) -> Stockage (BDD) -> Lecture (BDD) -> Affichage (Front).**

🛠️ 1. Pour vous : Le code du "Livre d'Or"

Voici le fichier `app.js` structuré et commenté. Il contient la connexion à la base de données et les deux routes nécessaires.

JavaScript

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

// Middlewares
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public')); // Pour le futur CSS

// 1. Connexion MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/livre_dor')
    .then(() => console.log("Connecté à MongoDB"))
    .catch(err => console.log(err));

// 2. Modèle de données
const Message = mongoose.model('Message', {
    pseudo: String,
    contenu: String,
    date: { type: Date, default: Date.now }
});

// 3. ROUTE GET : Lire les messages et afficher le formulaire
app.get('/', async (req, res) => {
    // On récupère TOUS les messages en BDD (le await est crucial !)
    const messages = await Message.find().sort({ date: -1 }); // Triés du plus
récent au plus vieux

    // On construit le HTML (en DWWM on utilisera plus tard un moteur comme
EJS ou React)
    let listeMessages = messages.map(msg => `
        <div style="border-bottom: 1px solid #ccc; margin-bottom: 10px;">
            <strong>${msg.pseudo}</strong> ($
{msg.date.toLocaleDateString()}) :
            <p>${msg.contenu}</p>
        </div>
    `).join('');

    res.send(`
        <h1>Le Livre d'Or</h1>
        <form action="/add-message" method="POST">
```

```
            <input type="text" name="pseudo" placeholder="Votre nom" required><br>
            <textarea name="contenu" placeholder="Votre message" required></textarea><br>
            <button type="submit">Envoyer</button>
        </form>
        <hr>
        <h2>Messages précédents</h2>
        ${listeMessages}
    `);
});

// 4. ROUTE POST : Enregistrer un nouveau message
app.post('/add-message', async (req, res) => {
    try {
        const nouveauMessage = new Message({
            pseudo: req.body.pseudo,
            contenu: req.body.contenu
        });
        await nouveauMessage.save();
        res.redirect('/'); // On recharge la page d'accueil pour voir le message
    } catch (err) {
        res.status(500).send("Erreur lors de l'enregistrement");
    }
});

app.listen(3000);
```

📘 2. La Fiche Pédagogique : Le Cycle de Vie des Données

📝 Fiche : Créer une fonctionnalité de A à Z

1. Le Schéma (Database Design) : Avant de coder, on définit ce qu'on stocke. Un message = un auteur + un texte + une date.

2. `Message.find()` : C'est l'équivalent du `SELECT * FROM messages` en SQL. L'option `.sort({ date: -1 })` permet d'avoir les messages frais en haut.

3. `res.redirect('/')` : C'est une notion capitale. Après un formulaire réussi, on ne reste pas sur la page de traitement, on redirige l'utilisateur vers la page de lecture.

4. L'Asynchronisme :

- On utilise `await` devant `find()` et `save()` car la base de données est un service extérieur qui répond avec un léger délai.

🧠 Test de compréhension final

1. **Pourquoi** ai-je ajouté `.sort({ date: -1 })` dans la route GET ? Que se passerait-il si je l'enlevais ?
2. **Modification :** Si vous vouliez ajouter un champ "Note sur 5" au livre d'or, quels sont les **3 endroits** dans le code où vous devriez faire une modification ?
3. **Analyse :** Quel est l'avantage d'utiliser `res.redirect('/')` au lieu de simplement faire `res.send('Message envoyé')` ? (Pensez à l'expérience utilisateur).

🎓 Prochaine étape ?

Vous avez maintenant les bases solides pour enseigner les fondamentaux du Back-end Node.js.

Comment ranger tout ce code dans des dossiers séparés (`models/`, `routes/`, `controllers/`) ? C'est la structure standard attendue pour le titre professionnel DWWM.

C'est l'étape ultime pour passer d'un "petit script" à une **application professionnelle**. En DWWM, on appelle cela le **Pattern MVC** (Modèle-Vue-Contrôleur).

L'idée est simple : un fichier ne doit avoir qu'une seule responsabilité.

🏗️ 1. Pour vous : L'architecture MVC

Voici comment nous allons réorganiser votre projet "Livre d'Or" pour qu'il soit propre et évolutif.

La nouvelle structure des dossiers

Plaintext

```
livre-dor/
 ├─── models/
 |    └─── Message.js      (Définit la structure des données)
```

```
├── controllers/
│   └── messageCtrl.js  (Contient la logique métier/fonctions)
├── routes/
│   └── messageRoutes.js (Définit les URLs)
├── public/            (Fichiers statiques)
├── .env               (Variables secrètes : mot de passe BDD)
└── app.js             (Point d'entrée, configure le serveur)
```

Étape 1 : Le Modèle (models/Message.js)

JavaScript

```javascript
const mongoose = require('mongoose');

const messageSchema = mongoose.Schema({
    pseudo: { type: String, required: true },
    contenu: { type: String, required: true },
    date: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Message', messageSchema);
```

Étape 2 : Le Contrôleur (controllers/messageCtrl.js)

On y met les fonctions `async` que nous avions dans les routes.

JavaScript

```javascript
const Message = require('../models/Message');

exports.getMessages = async (req, res) => {
    const messages = await Message.find().sort({ date: -1 });
    res.json(messages); // Ici on renvoie juste du JSON pour l'exemple
};

exports.createMessage = async (req, res) => {
```

```
    const nouveau = new Message({ ...req.body });
    await nouveau.save();
    res.status(201).json({ message: 'Objet enregistré !' });
};
```

Étape 3 : Le Routeur (routes/messageRoutes.js)

JavaScript

```
const express = require('express');
const router = express.Router();
const messageCtrl = require('../controllers/messageCtrl');

router.get('/', messageCtrl.getMessages);
router.post('/add', messageCtrl.createMessage);

module.exports = router;
```

📘 2. La Fiche Pédagogique : Pourquoi séparer le code ?

📝 Fiche : L'Architecture Modulaire

Le problème du "Code Spaghetti" :

Au début, mettre tout dans app.js est facile. Mais si vous avez 50 tables et 200 routes, le fichier devient illisible et impossible à maintenir en équipe.

La solution MVC :

- **M (Modèle) :** Représente les données. C'est le seul qui parle à la base de données.
- **V (Vue) :** Ce que l'utilisateur voit (HTML/React).
- **C (Contrôleur) :** Le cerveau. Il reçoit la requête, demande les données au Modèle, et les renvoie à la Vue.

Avantages pour un développeur :

1. **Lisibilité :** On sait exactement où aller pour modifier une route ou un schéma.
2. **Réutilisabilité :** On peut utiliser le même Modèle pour plusieurs contrôleurs.
3. **Travail en équipe :** Un développeur peut travailler sur les routes pendant qu'un autre crée les modèles.

🧠 Test de compréhension final (Bilan)

1. **Architecture :** Si je veux ajouter une validation pour que les messages fassent au moins 10 caractères, dans quel dossier vais-je principalement travailler ?
2. **Import/Export :** En Node.js, quelle instruction utilise-t-on pour rendre une fonction disponible dans un autre fichier ?
3. **Logique :** Pourquoi `app.js` doit-il rester le plus court possible ?

🏁 Conclusion de votre initiation

Félicitations ! Vous avez parcouru tout le spectre du développement Back-end avec Node.js :

- ✅ Installation et environnement (`npm`, `nodemon`).
- ✅ Création d'un serveur et gestion des routes (Express).
- ✅ Utilisation de middlewares (Sécurité, Log, Parsing).
- ✅ Gestion des méthodes HTTP (GET, POST, redirection).
- ✅ Connexion à une base de données NoSQL (MongoDB/Mongoose).
- ✅ Structuration professionnelle (MVC).

Voici le fichier `app.js` complet et structuré pour votre projet de **Livre d'Or**.

Une version **"Tout-en-un"** (plus simple pour débuter) et les commentaires nécessaires pour faire le lien avec la structure **MVC** que nous avons vue.

📄 Fichier `app.js` (Livre d'Or complet)

JavaScript

```
/**
 * PROJET : LE LIVRE D'OR (DWWM)
 * Objectif : Apprendre le CRUD (Create, Read) avec Node, Express et MongoDB
 */
```

```
const express = require('express');
const mongoose = require('mongoose');
const path = require('path');

const app = express();
const PORT = 3000;

// --- 1. CONFIGURATION DES MIDDLEWARES ---

// Permet de récupérer les données envoyées par les formulaires (req.body)
app.use(express.urlencoded({ extended: true }));

// Permet de servir les fichiers statiques (CSS, Images, JS client) depuis le
dossier 'public'
app.use(express.static(path.join(__dirname, 'public')));


// --- 2. CONNEXION À LA BASE DE DONNÉES ---

// Remplacez 'localhost' par '127.0.0.1' pour éviter certains délais sur
Windows
mongoose.connect('mongodb://127.0.0.1:27017/livre_dor_db')
    .then(() => console.log("✅ Connexion à MongoDB réussie !"))
    .catch((err) => console.error("❌ Erreur de connexion :", err));


// --- 3. MODÈLE DE DONNÉES (Le "M" de MVC) ---

const messageSchema = new mongoose.Schema({
    pseudo: { type: String, required: true },
    contenu: { type: String, required: true },
    date: { type: Date, default: Date.now }
});

const Message = mongoose.model('Message', messageSchema);


// --- 4. LES ROUTES (Le "C" et "R" de CRUD) ---

/**
 * ROUTE GET '/'
 * But : Récupérer tous les messages et afficher la page d'accueil
```

```
 */
app.get('/', async (req, res) => {
    try {
        // On récupère tous les messages du plus récent au plus ancien
        const messages = await Message.find().sort({ date: -1 });

        // Construction simplifiée du HTML (à remplacer plus tard par un moteur
de template comme EJS)
        let htmlContent = `
            <h1>📖 Livre d'Or</h1>
            <form action="/add-message" method="POST" style="margin-bottom:
30px;">
                <input type="text" name="pseudo" placeholder="Votre pseudo"
required style="display:block; margin-bottom:5px;">
                <textarea name="contenu" placeholder="Votre message" required
style="display:block; width:300px; height:100px; margin-bottom:5px;"></
textarea>
                <button type="submit">Poster mon message</button>
            </form>
            <hr>
            <h2>Derniers messages :</h2>
        `;

        messages.forEach(msg => {
            htmlContent += `
                <div style="border-left: 4px solid #3498db; padding: 10px; margin-
bottom: 15px; background: #f9f9f9;">
                    <strong>${msg.pseudo}</strong> <small>le ${
msg.date.toLocaleString()}</small>
                    <p>${msg.contenu}</p>
                </div>
            `;
        });

        res.send(htmlContent);
    } catch (error) {
        res.status(500).send("Erreur lors de la récupération des messages.");
    }
});

/**
 * ROUTE POST '/add-message'
```

```
 * But : Recevoir les données du formulaire et les enregistrer en BDD
 */
app.post('/add-message', async (req, res) => {
    try {
        const nouveauMessage = new Message({
            pseudo: req.body.pseudo,
            contenu: req.body.contenu
        });

        // Sauvegarde asynchrone dans MongoDB
        await nouveauMessage.save();

        // Une fois enregistré, on redirige l'utilisateur vers l'accueil pour voir son
message
        res.redirect('/');
    } catch (error) {
        res.status(400).send("Erreur lors de l'enregistrement du message : " +
error.message);
    }
});


// --- 5. LANCEMENT DU SERVEUR ---

app.listen(PORT, () => {
    console.log(`🚀 Serveur démarré sur : http://localhost:${PORT}`);
});
```

📘 Fiche Pédagogique Finale : Le fonctionnement global

📝 Fiche : Résumé d'une application Node.js

1. **L'initialisation :** On importe les modules (express, mongoose) et on configure les middlewares pour que le serveur sache lire les données entrantes.
2. **Le Schéma :** C'est le contrat. Il garantit que chaque message dans la base de données aura toujours un pseudo et un texte.
3. **L'Asynchronisme (async/await) :** La base de données est comme un service de livraison. On passe commande (find ou save), on attend que le livreur arrive (await), puis on continue le code.
4. **Le Cycle POST-REDIRECT-GET :** > * L'utilisateur envoie son message (**POST**).
    - Le serveur l'enregistre puis lui dit de changer de page (**REDIRECT**).

- Le navigateur demande la page d'accueil (**GET**).
- Le serveur renvoie la liste mise à jour.
- *Cela évite que l'utilisateur reposte le même message s'il rafraîchit la page !*

🚀 Prochaine étape pour vous

Ce fichier est parfait pour une démonstration en direct. Pour aller plus loin, vous pourriez essayer de :

- D'ajouter un bouton "Supprimer" à côté de chaque message (Route `app.delete`).
- D'ajouter du CSS dans le dossier `public` pour rendre l'interface plus moderne.

🛠️ La solution pour rétablir l'affichage

Assurez-vous que votre route `app.get('/')` ressemble exactement à ceci. J'ai ajouté des `console.log` stratégiques pour que vous puissiez suivre l'exécution dans le terminal :

JavaScript

```
app.get('/', async (req, res) => {
    try {
        console.log("1. Tentative de lecture de la base de données...");
        const messages = await Message.find().sort({ date: -1 });

        console.log(`2. Nombre de messages trouvés : ${messages.length}`);

        let listeMessagesHTML = "";
        messages.forEach(msg => {
            listeMessagesHTML += `
                <div style="border-bottom: 1px solid #ccc; padding: 10px;">
                    <strong>${msg.pseudo}</strong> : ${msg.contenu}
                </div>`;
        });

        console.log("3. Envoi du HTML au navigateur");
        res.send(`
            <h1>📖 Livre d'Or</h1>
            <form action="/add-message" method="POST">
```

```
            <input type="text" name="pseudo" placeholder="Pseudo" required><br>
            <textarea name="contenu" placeholder="Message" required></textarea><br>
            <button type="submit">Envoyer</button>
        </form>
        <hr>
        ${listeMessagesHTML}
    `);

  } catch (error) {
    console.log("❌ Erreur dans la route :", error);
    res.status(500).send("Erreur interne du serveur");
  }
});
```

🤔 Pourquoi ça ne s'affichait pas ? (Analyse pédagogique)

Si vous aviez mis le code de diagnostic **au-dessus** de votre route principale, Node.js s'arrêtait là :

1. Le navigateur demande `/`.
2. Express voit la première route qui correspond.
3. Express exécute `res.send("Le serveur répond")`.
4. **Le cycle est terminé.** Express ignore tout ce qui suit.

📘 Fiche Pédagogique n°9 : L'ordre des routes (Precedence)

📝 Fiche : Pourquoi l'ordre du code est vital ?

En PHP, vous appelez un fichier précis. En Node.js, vous appelez une URL qui traverse votre fichier `app.js` de haut en bas.

La règle d'or :

Express s'arrête à la première route qui correspond à l'URL demandée.

Exemple d'erreur courante :

JavaScript

```
app.get('/:page', ...); // Cette route est très large, elle "attrape" tout.
app.get('/admin', ...); // Celle-ci ne sera JAMAIS lue car la première a déjà répondu.
```

Conseil pour vos apprenants :

Rangez toujours vos routes de la plus spécifique à la plus générale.

🧠 Test de vérification

1. Modifiez votre code, enregistrez, et relancez le serveur.
2. Regardez votre terminal : voyez-vous passer les étapes 1, 2 et 3 ?
3. Si l'étape 2 n'apparaît jamais, c'est que `await Message.find()` bloque.

Est-ce que le formulaire apparaît enfin sur votre page ? Si oui, essayez de poster un message pour vérifier que la redirection fonctionne !